

Terraform + Claude — Learning Resources & Safe-Prompting Guide

Curated guides for building AWS infrastructure with Claude Code, plus how to prompt it safely.

Using an AI assistant (Claude Code) to write Terraform is a newer skill — most of the best material today is written guides rather than videos. This document collects those resources and, more importantly, gives you a practical cheat-sheet for driving Claude Code to build infrastructure safely. You've already been doing exactly this workflow throughout your practice project; this captures it so you can repeat it deliberately.

1. What 'Terraform + Claude' Actually Means

Instead of hand-writing HCL (Terraform's language) line by line, you describe the infrastructure you want in plain English, Claude Code generates the Terraform files, then you and Claude run the standard workflow together — format, validate, preview the plan, and apply only after you review it. Claude also debugs errors, applies best practices (encryption, tags, least privilege), and commits to GitHub.

The loop looks like this:

```
You:      'Create a Free-Tier EC2 instance in us-west-2 with a web server.'
Claude: writes versions.tf / providers.tf / variables.tf / main.tf / outputs.tf
Claude: runs 'terraform fmt' and 'terraform validate'
You:      run 'terraform plan' and READ it
You:      run 'terraform apply' -> type 'yes'
Claude: helps debug any error, then commits + pushes to GitHub
```

This is precisely how your S3, EC2, IAM, DynamoDB, and Lambda modules were built in this project.

2. Best Hands-On Written Guides

- **Deploy Infrastructure with Claude Code and Terraform (Hands-On)** — <https://computingforgeeks.com/claude-code-terraform-guide/>
- **Claude Code × Terraform: Complete AWS Automation Guide** — <https://claudelab.net/en/articles/claude-code/claude-code-terraform-aws-infrastructure-automation-guide>
- **Claude Code for Terraform & IaC — Modules, AWS, State** — <https://claude-code-workflows-henna.vercel.app/claude-code-terraform.html>

- **How to use Claude Code with Terraform, without hitting a wall** — <https://stategraph.com/blog/claude-code-terraform>
- **Generate Terraform Code with AI: Claude + GitHub MCP** — <https://www.jannidar.com/blog/mcp-github-claude-tf>
- **Terraforming the Cloud — AWS Infrastructure with AI as Copilot (Medium)** — <https://medium.com/@bte8287/day-1-terraforming-the-cloud-spinning-up-aws-infrastructure-with-ai-as-your-copilot-a10306ee82ee>
- **Claude AI for IaC: Safe Terraform & CloudFormation** — <https://www.blockchain-council.org/claude-ai/claude-ai-for-infrastructure-as-code-iac-terraform-cloudformation-safely/>

3. Finding Videos on YouTube

Dedicated high-view videos on this exact combo are still emerging. Search these phrases and pick a recent result with good views:

- "Claude Code Terraform AWS"
- "AI generate Terraform infrastructure Claude"
- "Claude Code DevOps infrastructure as code"
- "vibe coding infrastructure Terraform AI"

Channels most likely to publish quality versions: freeCodeCamp, TechWorld with Nana, and newer AI-DevOps creators. For Terraform fundamentals themselves (not AI-assisted), TechWorld with Nana's 'Complete Terraform Course' remains the top pick.

4. Cheat-Sheet: Prompting Claude Code to Build Terraform Safely

This is the most valuable part. Follow these habits and Claude Code becomes a reliable infrastructure partner instead of a risky one.

4.1 Give context up front

- **Always state:** Name the cloud (AWS), the region (us-west-2), and that you want Free-Tier where possible.
- **Ask for structure:** Ask for a modular structure (modules/ + a root that wires them) so it scales, like your terraform-aws-practice project.
- **Demand security:** Ask for secure defaults explicitly: encryption on, public access blocked, least-privilege IAM, tags.

Prompt example: *"Create a modular Terraform setup for an S3 bucket in us-west-2 — versioning on, encryption at rest, all public access blocked, and least-privilege access. Put it in a reusable module and wire it into a root project."*

4.2 Enforce the safe workflow

- **Validate first:** Ask Claude to run 'terraform fmt' and 'terraform validate' BEFORE anything touches AWS.
- **Review the plan:** Insist YOU run (or at least review) 'terraform plan' before 'apply'. Never let apply happen without reading the plan.
- **You own apply/destroy:** Tell Claude not to run 'terraform apply' or 'destroy' against your account on its own — you run those consciously.

Prompt example: *"Write and validate the code, but I'll run terraform plan and apply myself so I can review what changes."*

4.3 Protect secrets and cost

- **Never commit secrets:** Tell Claude to add a .gitignore for state files, tfvars, and .pem keys — and to never commit credentials.
- **Keys stay local:** Never paste AWS access keys into the chat. Configure them locally with 'aws configure' yourself.
- **Stay free:** Ask for Free-Tier-eligible resources and a reminder to 'terraform destroy' when done.

Prompt example: *"Make sure state files, tfvars, and any keys are gitignored, and keep everything Free-Tier eligible."*

4.4 Let Claude debug errors

- **Paste errors verbatim:** When 'terraform plan/apply' errors, paste the FULL error back to Claude. It will identify the layer (credentials, config, or a service limit), query AWS for ground truth, fix the specific cause, and re-validate.

Prompt example: *"terraform apply failed with: <paste the exact error>. What's the cause and the fix?"*

4.5 Ask for explanations, not just code

- **Learn as you go:** You're learning — ask Claude WHY it chose something (a distribution style, a GSI, a role vs a user). That turns code generation into education.

Prompt example: *"Explain why you used a GSI on customer_id and what would break without it."*

5. The Golden Rules (Memorize)

1. Always read 'terraform plan' before 'apply' — it's your one chance to catch a mistake.
2. You run apply and destroy, not the AI — those change/charge your live account.
3. Never paste secrets into chat; configure credentials locally.

4. Gitignore state, tfvars, and keys — never commit them.
5. Stay Free-Tier and destroy when done practicing.
6. Ask 'why', not just 'how' — you're building skills, not just resources.

6. You've Already Done This

Every service in your terraform-aws-practice project — S3, EC2, IAM, DynamoDB, and the order-processing Lambda — was built with this exact Terraform + Claude workflow, including debugging real errors (invalid credentials, AMI volume size, Free-Tier instance type, globally-unique bucket names, and adding S3-read permission for the Lambda trigger). Re-reading your Terraform_AWS_Practice_Playbook.pdf alongside this guide is the best 'tutorial' you have, because it's your own hands-on record of the workflow these articles describe.